

Atomic Composite Mode - One Signature, All-or-Nothing

Status: LIVE on mainnet. A multi-step intent such as "swap 2 SUI to USDC then lend it on NAVI" or "send 1 SUI to alice and send 1 SUI to bob" can execute as **one Programmable Transaction Block (PTB), one wallet signature**, all-or-nothing - instead of multiple separate signed steps.

This is Dewlock's headline chaining capability and a direct expression of the moat: **the LLM only proposes the legs; a deterministic builder composes them and the Guardian re-derives and fail-closes before any signature is requested.**

1 - What the user does

1. Type a compound intent: swap 2 SUI to USDC then lend it on navi Or send 1 SUI to @alice and send 1 SUI to @bob.
2. A **chain-plan card** appears with multiple steps and a toggle: "Run as 1 transaction (atomic)".
3. Click it -> a single tx-preview card renders with the full **flow map** (all legs, real protocol logos, estimated output).
4. Sign **once** in the wallet -> all legs settle together, or neither does.

The atomic toggle is offered only when the plan is **eligible**: 2-8 steps, all using allowlisted action types (send, swap, lend_deposit, stake), optional chaining (a leg may consume a prior leg's output), and nothing has been signed yet.

2 - How one PTB is built (dynamic recipe)

The composite builder (packages/sui/src/build-composite.ts -> buildDynamicComposite) assembles an ordered sequence of legs into a single PTB, with optional structural chaining:

Component	What happens	Why it matters
Recipe build	buildDynamicRecipe(legs) creates a CompositeRecipe with id="dynamic"; allowed MoveCall targets per leg come from the existing per-action allowlist (allowedTargetsForLegType)	Closed registry: only allowlisted actions compose; ad-hoc composition is refused
Leg 0	First leg is built independently (no input from a prior leg)	Establishes the entry point (e.g., a swap from the sender's wallet, a send from the sender, a stake of sender's SUI)

Leg k (k >= 1)	If leg k's <code>amountFrom="prev-output"</code> , consume leg k-1's output coin structurally (no wallet round-trip); else build independently from the sender's wallet	Chaining is opt-in per leg; independent legs do NOT consume prior output
Coin output handling	After each leg builds, if the next leg doesn't consume its output, return it to the sender via <code>transferObjects([outputCoin], sender)</code>	No coin object dangles mid-PTB; unconsumed outputs go back to the user
Coverage gate (swap legs only)	<code>assertSuiGasCoverage</code> - total SUI >= swap amount + 0.05 SUI gas reserve	A shortfall is caught before the route fetch, surfaced as the same <code>insufficient_gas</code> gate as a normal swap
Route (swap legs only)	Cetus aggregator <code>findRouters</code> (venues: CETUS + DEEPBOOK) -> best route + estimated output	Same engine as a normal swap, so multi-path routes compose cleanly

Atomicity: a Move abort in *any* leg reverts the **entire** PTB. Nothing executes on a partial failure.

Example flows: - **swap->lend**: swap produces a coin; lend consumes it structurally. No wallet round-trip. - **send->send**: two independent sends from the sender's wallet. Both settle together. - **swap->stake**: swap produces a coin; stake deposits it. Full chaining. - **send+swap+lend** (mixed, same priority): three independent legs (each from the wallet) settling together. Each renders as a separate outflow in the flow map.

Why Cetus for swaps, not Aftermath: Aftermath's `addTransactionForCompleteTradeRoute` splits the gas-token SUI during a deferred resolution step that aborts (MoveAbort 46001) for multi-path routes - so atomic *always* fell back. The Cetus aggregator returns the output coin directly and is the same route engine normal swaps already use.

3 - The Guardian re-verifies the whole composite

The builder produces bytes; it does **not** grant trust. `checkCompositeRecipe` (`packages/agent/src/guardian.ts`) re-derives four invariants on the **built PTB** before returning an `approvedDigest`:

1. **Closed-recipe registry** - only recipes in the registry (including the dynamic recipe) are composable. Ad-hoc composition is refused.
2. **Target multiset** - every MoveCall in the PTB must be in the recipe's `allowedTargets` (per-action allowlists: aggregator swaps, NAVI/Suilend lend calls, Haedal staking). Send legs emit ZERO MoveCalls (`TransferObjects` is a PTB command, not a MoveCall), so send legs are identified by the absence of calls in their range. `allowSignatureMatch` admits aggregator per-hop calls by `module::function` signature. **Every declared leg must be present** (or for sends, the range must have zero calls and matching balance deltas).

3. **Coin-type linkage** (chained legs only) - for legs where `amountFrom="prev-output"`, the output coin type of the prior leg must equal the input coin type of the consuming leg (the structural proof the legs are connected).
4. **Recipient-aware anti-leak + dual caps** - via `checkCompositeDeltaAntiLeak`: - **When there are declared send legs**: Every third-party inflow (positive balance delta) in the dry-run must exactly match a declared send leg's (recipient, coinType, amount) - multiset equality. Any inflow not covered by a declared leg -> BLOCK. Coin objectChanges with third-party owners are safely skipped (they appear alongside balanceDeltas; balanceDeltas is authoritative). - **When there are NO send legs**: Zero tolerance for third-party balance inflows or third-party objectChange owners. - **Caps**: USD value within `TX_USD_CAP` + net-SUI outflow within `NET_SUI_DELTA_CAP_MIST`.

Any failure -> **BLOCK** (no signature). The approved digest is bound to the exact bytes (**WYSIWYS**): if the bytes change after approval, signing is refused.

4 - The flow map - topology reflects chaining

The composite tx-preview renders the legs by chaining: independent legs branch straight from "You" (the wallet) as separate outflows; only a chained leg (consuming a prior leg's output) connects to the prior protocol node.

Example: swap->lend (chained):

```
[You] - 2 SUI -> [Cetus Aggregator - Swap -> ~6.2 USDC] - ~6.2 USDC -> [NAVI - Lending - deposit]
```

Example: send->send (independent):

```
[Alice]
← 1 SUI
[You]
-> 1 SUI
[Bob]
```

Example: swap+send+lend (mixed, all independent):

```
[You] +-> 2 SUI -> [Cetus - swap]
      +-> 1 SUI -> [Alice]
      +-> 50 USDC -> [NAVI - lend]
```

- **Real protocol logos** on each node (Cetus, NAVI, Haedal, etc.), resolved from the bundled brand assets.
- **Estimated output** from the live route shown on action nodes (swap output on the swap node, expected haSUI output on the stake node) and on the edge connecting to the consuming leg.
- The estimate is the engine's own number (e.g., the aggregator's route number for swaps); the **minimum** is the slippage floor (for swaps) or floor-based bound (for other actions) that actually executes.

5 - Graceful fallback (funds and safety never affected)

If the composite cannot be built or the Guardian blocks it, atomic **degrades to the sequential step-by-step chain** - the user signs each step in order, and every Guardian check still applies. The fallback message is classified honestly:

- **Balance/gas shortfall** -> *"Not enough SUI to swap that amount and still cover network gas. Reduce the amount or top up."* (Falling back would hit the same shortfall, so it doesn't pretend to help.)
- **Route can't be composed** -> *"Atomic bundling isn't available for this swap route right now ... falling back to step-by-step; your funds and every Guardian check are unaffected."*

Sequential mode also auto-recovers from a transient stale-object error by rebuilding the step with fresh bytes (bounded retries; stale bytes are never re-sent, so a coin is never equivocated).

6 - Scope & limitations

- **Allowlisted actions only** - composable actions are send, swap, lend_deposit, stake (haSUI only; afSUI builds its own tx and is not composable). Any other action type in a composite -> BLOCK.
- **1-8 legs** - DoS/UX bound. Enforced by `buildDynamicRecipe`.
- **Page refresh** loses an in-flight sequential chain (durable resume not yet implemented). The atomic path is a single signature, so it isn't affected mid-flight.
- The **estimate** is a dry-run figure; the guaranteed amount is the slippage-floor minimum (for swaps) or the safe floor bound (for other actions).

7 - Why this is safe (the moat, restated)

- The LLM proposes **legs**, never bytes. The deterministic builder composes them; the Guardian re-derives the full PTB and fail-closes.
- **One signature, WYSIWYS** - the user signs the exact approved bytes; any change after approval is refused.
- **All-or-nothing** - a partial failure reverts everything.
- **Zero user-fund keys server-side** - Dewlock never holds spend authority; the wallet signs the single composed PTB.

Related: `system-architecture.md` (full Guardian gate list), `copilot-command-guide.md` (end-user command reference).